第4章作业：4.8　4.10　4.30　4.31

第5章作业预告：　5.18 5.30

第5章　数组和广义表（Arrays & Lists）

5.1　数组的定义

5.2 数组的顺序存储表示和实现

5.3 矩阵的压缩存储

5.4 广义表的定义

5.5 广义表的存储结构

二、稀疏矩阵的操作（以转置运算为例，加减用十字链表）

目的：

$$M=\begin{pmatrix} 0 & 12 & 9 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ -3 & 0 & 0 & 0 & 14 & 0 \\ 0 & 0 & 24 & 0 & 0 & 0 \\ 0 & 18 & 0 & 0 & 0 & 0 \\ 15 & 0 & 0 & -7 & 0 & 0 \end{pmatrix} \xrightarrow{\text{转置后}} \begin{pmatrix} 0 & 0 & -3 & 0 & 0 & 15 \\ 12 & 0 & 0 & 0 & 18 & 0 \\ 9 & 0 & 0 & 24 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & -7 \\ 0 & 0 & 14 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \end{pmatrix} T$$

已知三元组表 a.data

(1, 2, 12)
(1, 3, 9)
(3, 1, -3)
(3, 5, 14)
(4, 3, 24)
(5, 2, 18)
(6, 1, 15)
(6, 4, -7)

求三元组表 b.data

(1, 3, -3)
(1, 6, 15)
(2, 1, 12)
(2, 5, 18)
(3, 1, 9)
(3, 4, 24)
(4, 6, -7)
(5, 3, 14)

提问：**若采用三元组压缩技术存储稀疏矩阵，只要把每个元素的行下标和列下标互换，就完成了对该矩阵的转置运算，这种说法正确吗？**

答：**肯定不正确！**

除了： **（1）每个元素的行下标和列下标互换（即三元组中的i和j互换）；**

还需要： **（2） T的总行数mu和总列数nu也要互换；**

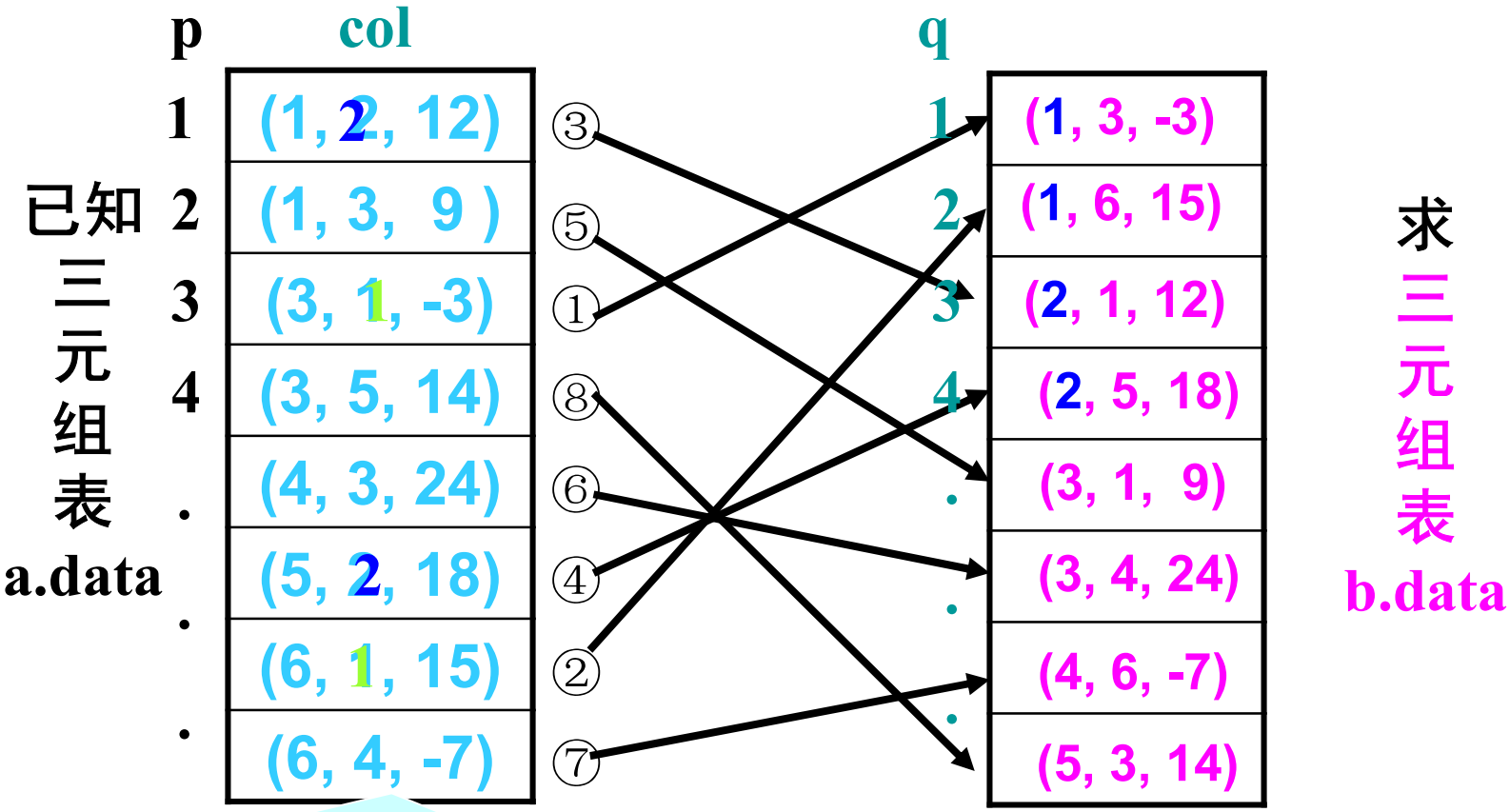
（3）重排三元组内各元素顺序，使转置后的三元组也按行（或列）为主序有规律的排列。

上述（1）和（2）容易实现，难点在（3）。

有两种实现转置的方法
- 压缩转置
- 快速(压缩)转置

方法1：压缩转置

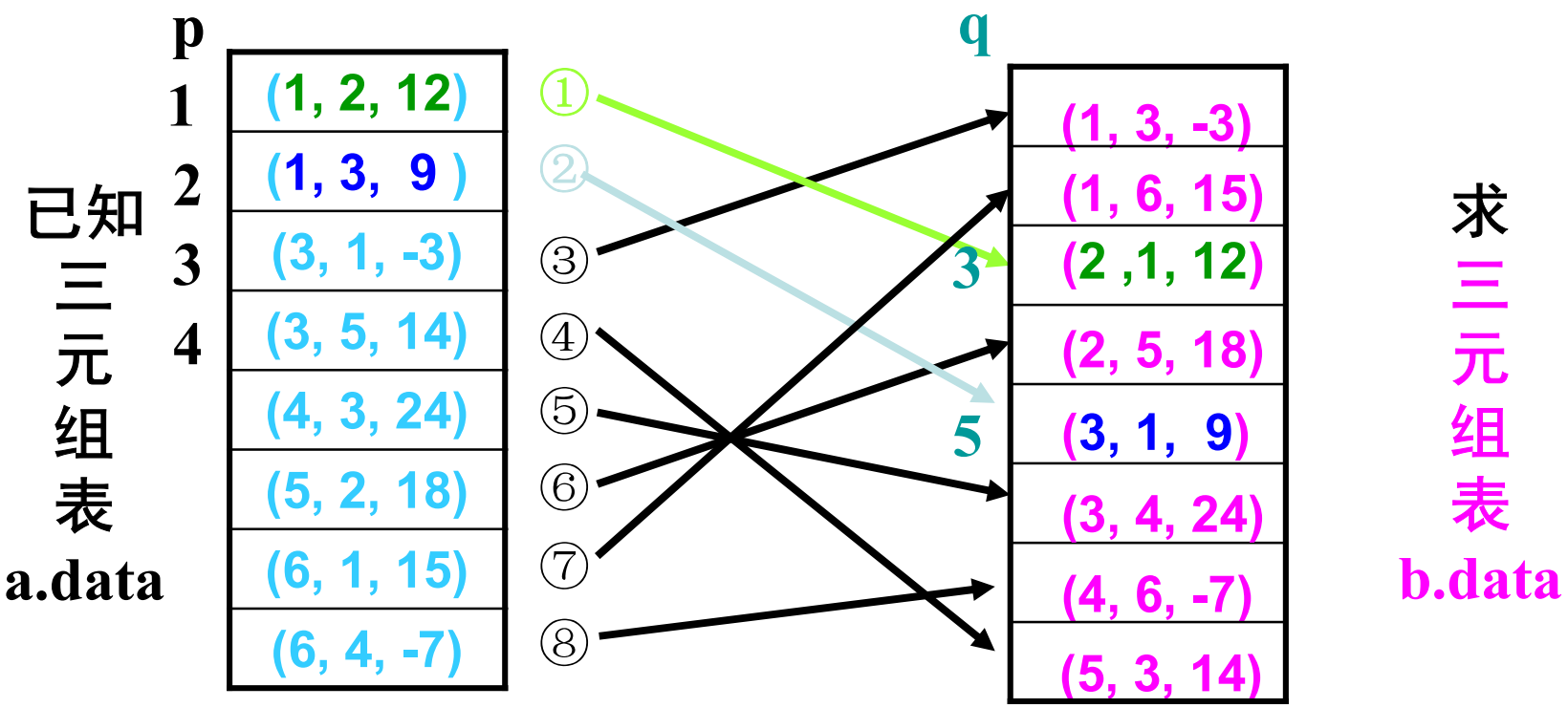
思路：反复扫描a表（记为a.data）中的列序，从j=1～n依次进行转置。



每个元素的列分量表示为： **a.data[p].j**

方法2　快速转置

思路：依次把a.data中的元素直接送入b.data的恰当位置上（即M三元组的p指针不回溯）。



关键：怎样寻找b.data的“恰当”位置?

设计思路：

如果能预知M矩阵每一列(即T的每一行)的非零元素个数，
又能很快得知第一个非零元素在b.data中的位置，
则扫描a.data时便可以将每个元素准确定位(因已知若干参考点)

请注意a.data特征：每列首个非零元素必定先被扫描到。

技巧： 为实现转置运算，应当按列生成 M 矩阵三元组表的两个辅助向量，让它携带每列的非零元素个数 NUM(i) 以及每列的第一个非零元素在三元组表中的位置POS(i) 等信息。

辅助向量的样式：

i	1	2	3	4	5	6
NUM(i)	2	2	2	1	1	0
POS(i)	1	3	5	7	8	9

计算式：POS(1)＝1

POS(i)＝POS(i-1)+NUM(i-1)

令：M矩阵中的列变量用col表示；

num[col]：存放M中第col 列中非0元素个数

cpos[col]：存放M中第col列的第一个非0元素的位置

(即b.data中待计算的“恰当”位置所需参考点)

col	1	2	3	4	5	6
num[col]	2	2	2	1	1	0
cpos[col]	1	3	5	7	8	9

计算式：cpot(1)=1

想一想：是从原始矩阵M中来统计num[col]呢，还是从对应的三元组表a.data中统计更合理?

讨论：求出按列优先的辅助向量后，如何实现快速转置?

由a.data中每个元素的列信息，可以直接从辅助向量cpos[col]中查出在b.data中的“基准”位置，进而得到当前元素之位置。

三元组表 a.data

p/t	col
1	(1, 2, 12)
2	(1, 3, 9)
3	(3, 1, -3)
4	(3, 5, 14)
.	(4, 3, 24)
.	(5, 2, 18)
.	(6, 1, 15)
.	(6, 4, -7)

快速转置算法描述：

```
Status FastTransposeSMatrix（TSMatirx M, TSMatirx &T）
{                    //M是顺序存储的三元组表，求M的转置矩阵T
T.mu = M.nu ;T .nu = M.mu ; T.tu = M.tu ;
 if ( T.tu ) {
for(col = 1; col <=M.nu; col++) num[col] =0;          //先清0,再
for( t = 1; t <=M.tu; t++) {col =M.data[t] .j ; ++num [col] ;}  //统计每列非零元素个数
 cpos[ 1 ] =1;                                        //再生成每列首元位置辅助向量
 for(col = 2; col <=M.nu; col++)  cpos[col ]=cpos[col-1]+num [col-1 ] ;
 for( p =1; p <=M.tu ; p ++ )     //p指向a.data，循环次数为非0元素总个数tu
  { col =M.data[ p ]. j ; q =cpos [ col ];//查辅助向量得q，即T中位置
            T.data[q].i = M.data[p]. j;
            T.data[q].j = M.data[p]. i;
            T.data[q]. value = M.data[p]. value;
            + + cpos[col] ;
  } //for
} //if
return OK;
} //FastTranposeSMatrix;
```

前3个for循环用来产生两个辅助向量

元素转置

重要！巧妙修改向量内容（列坐标加1），使其不再固定指向本列第一个非零元素，而是预备给同列的下一非零元素定位之用

快速转置算法的效率分析：

1. 与常规算法相比，附加了生成辅助向量表的工作。增开了2个长度为列长的数组(num[ ]和cpos[ ]）。
2. 从时间上，此算法用了4个并列的单循环，而且其中前3个单循环都是用来产生辅助向量表的。

```
for(col = 1; col <=M.nu; col++){ };  循环次数 = nu（列数）；
for( i = 1; i <=M.tu; i ++) { };     循环次数 = tu （非0元素个数）；
for(col = 2; col <=M.nu; col++) { }; 循环次数 = nu;
for( p =1; p <=M.tu ; p ++ ) { };    循环次数 = tu;
```

该算法的时间复杂度 = nu+tu+nu+tu=O(nu+tu)

讨论：最恶劣情况是矩阵中全为非零元素，此时tu=nu*mu而此时的时间复杂度也只是O(mu*nu)，并未超过传统转置算法的时间复杂度。

小结：传统转置：O(mu*nu)　压缩转置：O(mu*tu)
压缩快速转置：O(nu+tu)

增设辅助向量，牺牲空间效率换取时间效率。

稀疏矩阵相乘的算法略，见教材P101-103

第5章　数组和广义表（Arrays & Lists）

数组和广义表的特点：一种特殊的线性表

① 元素的值并非原子类型，可以再分解，表中元素也是一个线性表（即广义的线性表）。
② 所有数据元素仍属同一数据类型。

5.1 数组的定义

5.2 数组的顺序表示和实现

5.3 矩阵的压缩存储

5.4 广义表的定义

5.5 广义表的存储结构

5.4 广义表的定义

1、定义：

广义表是线性表的推广，也称为列表（lists）

记为： $LS = (a_1, a_2, \ldots\ldots, a_n)$

广义表名　表头(Head）　表尾 (Tail)　n是表长

在广义表中约定：

①用小写字母表示原子类型，用大写字母表示列表。

②第一个元素是表头，而其余元素组成的表称为表尾；

特别提示：任何一个非空表，表头可能是原子，也可能是列表；但表尾一定是列表！

讨论：广义表与线性表的区别和联系?

广义表中元素既可以是原子类型，也可以是列表；当每个元素都为原子且类型相同时，就是线性表。

2、特点：

- 有次序性　　**一个直接前驱和一个直接后继**
- 有长度　　**＝表中元素个数**
- 有深度　　**＝表中括号的重数**
- 可递归　　**自己可以作为自己的子表**
- 可共享　　**可以为其他广义表所共享**

例1：求下列广义表的长度。

1）A =( )　　n=0，因为A是空表

2）B = (e)　　n=1，表中元素e是原子

3）C =(a ,(b , c , d))　　n=2，a 为原子，(b,c,d)为子表

4）D=(A , B ,C)　　n=3，3个元素都是子表

5）E=(a, E)　　n=2，a 为原子，E为子表

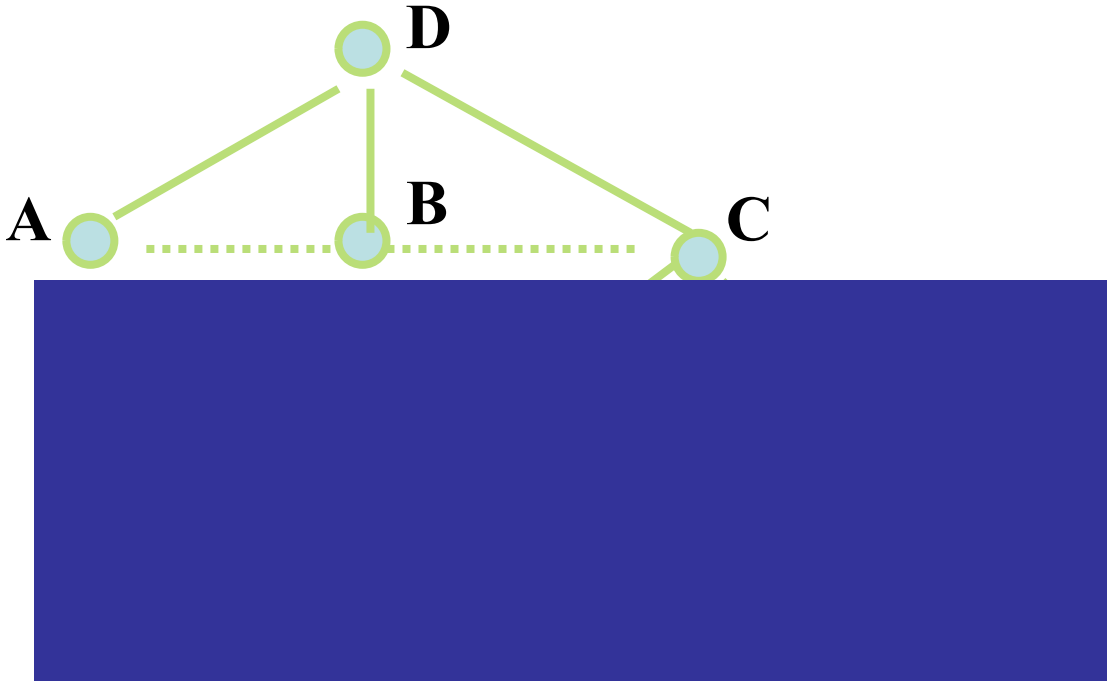
D=(A,B,C)=(( ),(e),(a,(b,c,d)))，共享表

E=(a,E)=(a,(a,E))= (a,(a,(a,…….)))，E为递归表

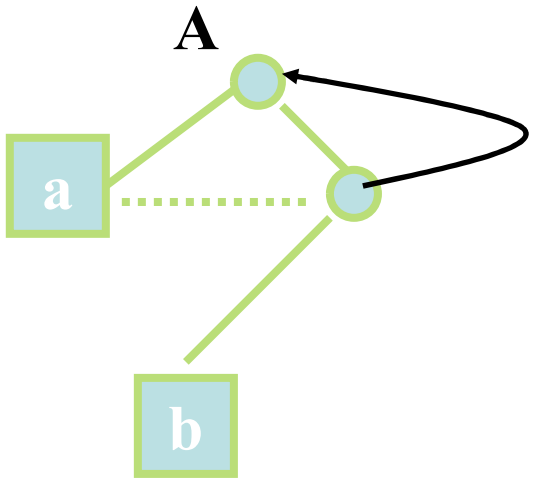
例2：试用图形表示下列广义表.

(设 ○ 代表子表， e 代表元素)

① D=(A,B,C)=(( ),(e),(a, (b,c,d)))



② A=(a , (b, A))



①的长度为3，
深度为3

②的长度为2，
深度为∞

深度 = 括号的重数 = ○结点的层数

广义表的抽象数据类型定义见教材P107-108

ADT Glist{

数据对象：D={ e_i | i=1,2,……,n;n≥0; e_i∈AtomSet或Glist }

数据关系：R1={ <e_{i-1},e_i> | e_{i-1},e_i ∈D, 2≤i ≤ n }

基本操作：

InitGList(&L);

操作结果：创建空的广义表L.

CreateGList(&L,S);

初始条件：S是广义表的书写形式串。

操作结果：由S创建广义表L.

……

}ADT Glist

介绍两种特殊的基本操作：

GetHead (L) ——取表头(可能是原子或列表)

初始条件：广义表L存在。

操作结果：取广义表L的头。

GetTail (L) ——取表尾(一定是列表)

初始条件：广义表L存在。

操作结果：取广义表L的尾。

例：求下列广义表操作的结果（严题集5.10②）

1. GetTail【(b, k, p, h)】＝ (k, p, h) ;

2. GetHead【((a,b), (c,d))】＝ (a,b) ;

3. GetTail【((a,b), (c,d))】＝ ((c,d)) ;

4. GetTail【 GetHead【((a,b),(c,d))】 】＝（b）;

(a,b)

5. GetTail【（e）】＝ ( ) ;

6. GetHead【(( ))】＝ ( ) .

7. GetTail【(( ))】＝ ( ) .

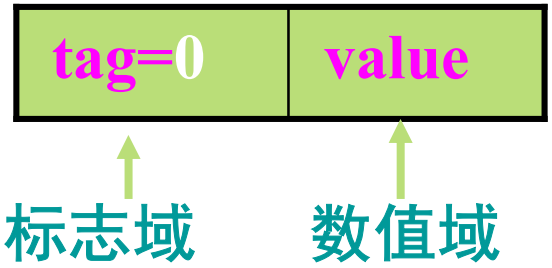
5.5 广义表的存储结构

由于广义表的元素可以是不同结构（原子或列表），难以用顺序存储结构表示，通常用链式结构，每个元素用一个结点表示。

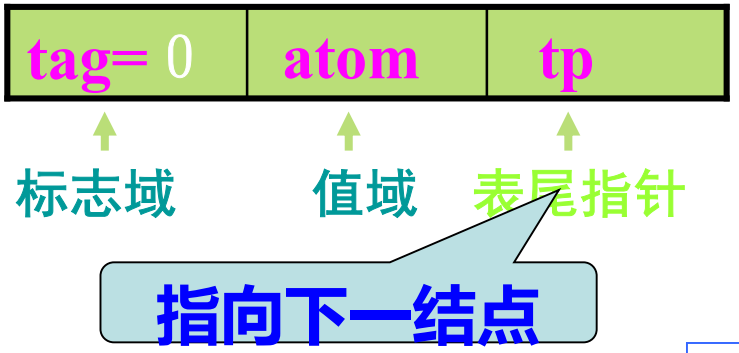
注意：列表的“元素”还可以是列表，所以结点可能有两种形式：

1. 原子结点：**表示原子，可设2个域或3个域，依习惯而选。**

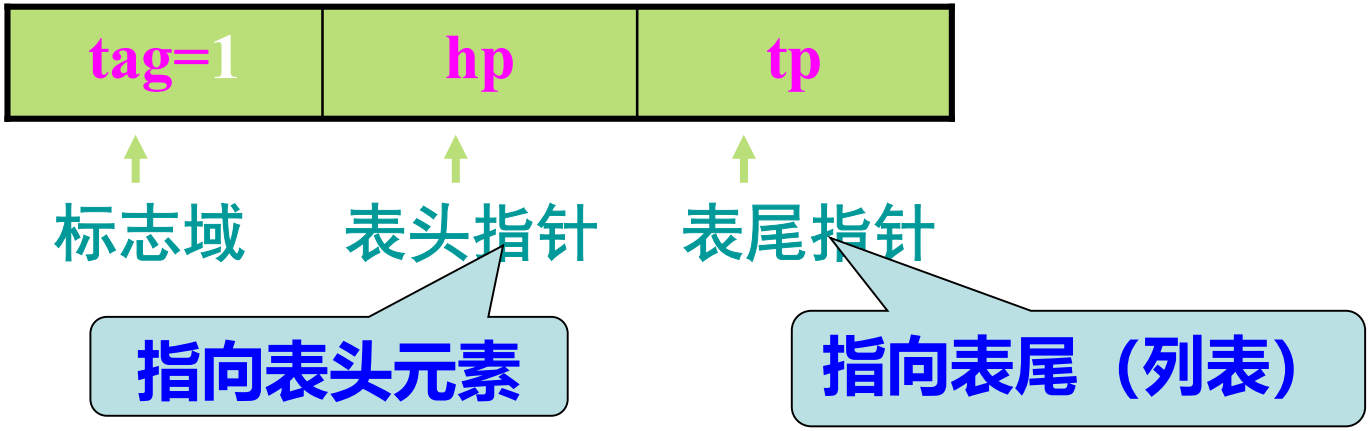
法1：标志域，数值域



法2：标志域、值域、表尾指针



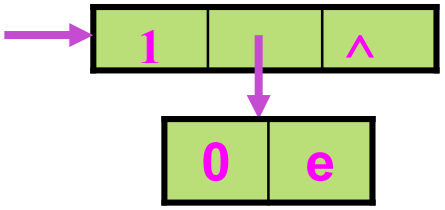
2. 表结点：表示列表，若表不空，则可分解为表头和表尾，用3个域表示：标志域，表头指针，表尾指针。



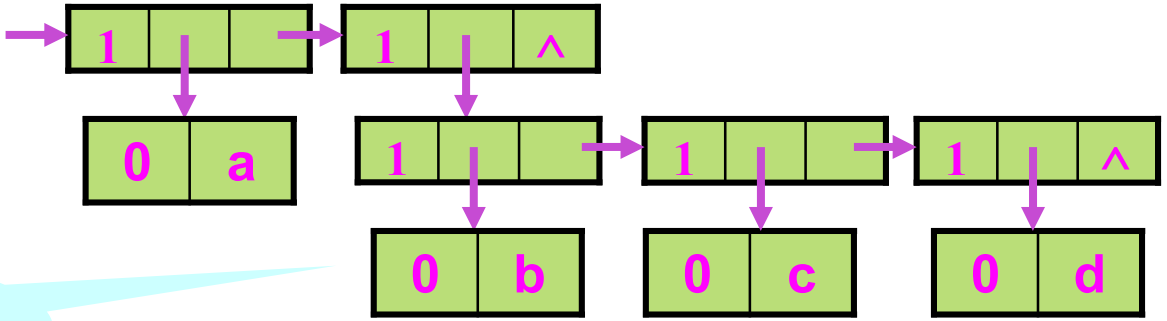
例：

① A =( )　　A=NULL

② B=(e)



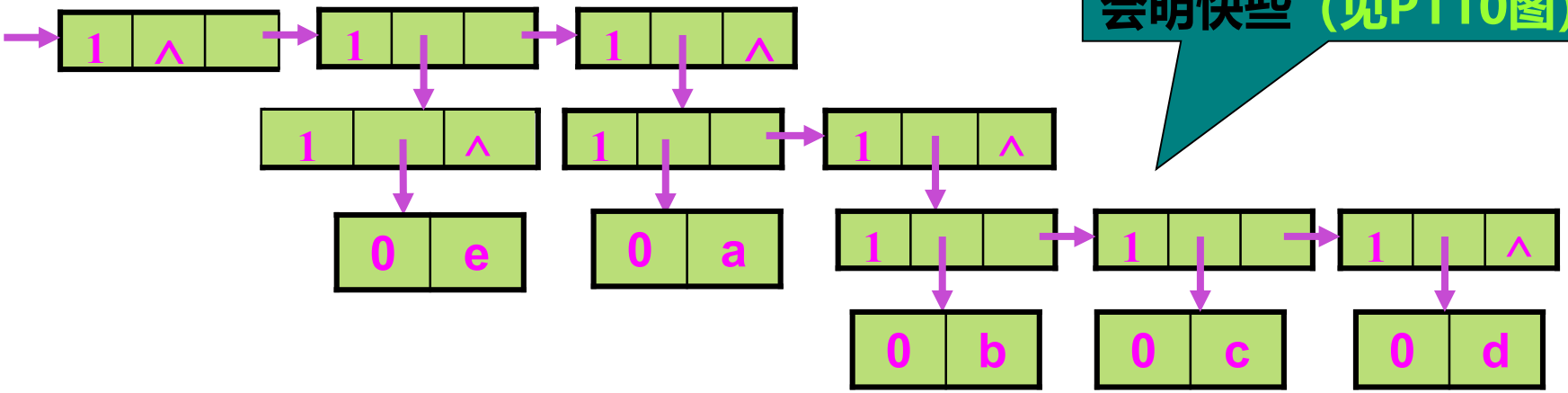
③ C =(a ,(b , c , d))



原子结点的第一种表示法

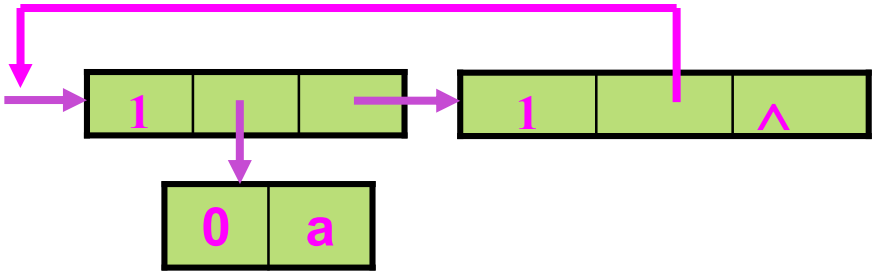
④ D=(A , B ,C)＝(( ),(e),(a,(b,c,d)))

原子结点的第一种表示法比较麻烦，若采用第二种会明快些（见P110图）



（参见教材P109图）

⑤ E=(a, E)



本章小结

1. 数组可视为一种广义线性表；
2. 数组的存储有行/低地址优先和列/高地址优先两种不同的顺序；
3. 对于稀疏矩阵，有较好的压缩存储和运算方法；
4. 广义表（列表）是线性表的推广，也是一种线性结构；
5. 任何一个非空表，表头可能是原子，也可能是列表；但表尾一定是列表。

第5章结束

线性结构内容结束